MakeoverArena Development Report

MakeoverArena — Development Report

Period: 28 May 2026 – 2 June 2026 (5 Days) **Prepared by:** Engineering Team **Project:** makeoverarena.com — Nigeria Study Abroad Consultancy Platform **Status:** Development Complete · Awaiting Credential Configuration · Ready for Deployment

---

Executive Summary

Over the past five days, the MakeoverArena digital platform was taken from a partially built skeleton to a fully production-ready web application. The platform now covers every aspect of the business lifecycle: marketing, lead capture, consultation booking, payment collection, document management, client tracking, admin operations, AI-powered chat support, and live WhatsApp communication.

The codebase comprises **89 TypeScript/TSX source files**, **15 API routes**, **11 database tables**, **9 email templates**, **4 Supabase Edge Functions**, and **1 real-time WhatsApp inbox**. Zero TypeScript errors. All 23 public routes return HTTP 200 in local testing.

The application cannot go live until the project owners supply credentials for eight third-party services. This report documents every feature built and provides exact, step-by-step instructions for obtaining and configuring each credential.

Part 1 — What Was Built

1.1 Public Marketing Website

The entire public-facing website was built and is production-ready. Every page uses a consistent navy/gold brand system with responsive mobile layouts.

Page	Route	Description
Homepage	`/`	Hero section, stats bar (2,400+ students, 92% success rate), service cards, country destinations, testimonials carousel, FAQ preview, CTA section
About	`/about`	Mission statement, company values, team profiles, 6-milestone company history
Undergraduate	`/services/undergraduate`	Full service page with programme details, requirements, success stats
Graduate/Masters	`/services/graduate`	MSc/MBA admissions service page
Scholarships	`/services/scholarships`	Scholarship identification and application service
Visa Support	`/services/visa`	Student visa guidance service page

Page	Route	Description
Success Stories	`/success-stories`	6 student testimonial cards with country and scholarship badges
FAQ	`/faq`	Accordion-style FAQ grouped in 6 categories with live search
Book Consultation	`/book`	Cal.com booking embed, office hours, WhatsApp and email contact
Privacy Policy	`/privacy`	GDPR-compliant 10-section privacy policy
Terms of Service	`/terms`	10-section terms covering services and liability
Cookie Policy	`/cookies`	3-category cookie disclosure with GDPR banner

Navbar: Responsive, transparent on homepage, sticky on all other pages, active route highlighting, Services dropdown, My Account link, mobile hamburger menu.

Footer: 5-column layout with brand tagline, contact details (phone + WhatsApp button), social links (Instagram, LinkedIn, Twitter/X), service links, destination list, legal links. All contact details are driven by environment variables — no hardcoding.

1.2 Student Application Form (5-Step Multi-Step Form)

Route: `/apply`

The application form is the primary lead generation tool. It captures everything a consultant needs before a first meeting.

Step 1 — Service Selection Six service cards: Undergraduate, Graduate/Masters, PhD, Scholarship, Visa Support, Not Sure Yet. Visual card-selection interface with gold checkmark on selection.

Step 2 — Personal Information Full name, email, phone number, WhatsApp (optional), current city, and country.

Step 3 — Academic Background Current education level (High School / Bachelor's / Master's / Other), field of study, GPA or percentage, expected graduation year.

Step 4 — Study Preferences Multi-select preferred countries (13 options), preferred intake period, annual budget range, standardised tests taken (IELTS, TOEFL, GRE, GMAT, SAT, ACT).

Step 5 — Final Details Referral source (Instagram, Google, referral, Facebook, WhatsApp, TikTok, YouTube, Other), referrer name if applicable, any additional notes, email opt-in checkbox.

Features built into the form: - Zod v4 validation on every field, per step — prevents bad data reaching the API - localStorage draft save — if the browser is closed mid-form, progress is fully restored on return - Rate limiting — 5 submissions per hour per IP address via Upstash Redis (graceful fallback if Redis is unavailable) - Success screen after submission — shows next steps, buttons to Book Consultation, Create Account, or Return to Home - Google Analytics event tracking — `form_start`, `form_step_complete`, `form_submit` events - Signup link pre-fills student's email on the account creation page

---

1.3 Client Account System

Students can create and log into personal accounts to track their journey.

Route	Function
`/signup`	Account creation with full name, email, password. Accepts `?email=` prefill from apply form. Sends Supabase Auth confirmation email
`/login`	Email/password login. Accepts `?email=` and `?redirect=` query params. Includes password reset by email
`/dashboard`	Authenticated client dashboard (see below)

Client Dashboard (`/dashboard`) — Tab-based interface with six sections:

1. **Overview** — KPI tiles (applications count, consultations count, total paid, outstanding balance), upcoming consultation with Join Meeting button, payment due alert
2. **My Profile** — Editable personal and academic details; read-only fields (email, service type, education level) are protected from client tampering
3. **Applications** — Full application tracking with stage-by-stage progress bar, institution and country, assigned advisor name
4. **Consultations** — Full history with Join Meeting links for upcoming sessions
5. **Payments** — Payment history with Paystack "Pay Now" button on pending items, running totals for paid and outstanding
6. **Documents** — File upload with document type selection (transcript, passport, SOP, LOR, CV, test scores, financial, certificate, ID, other), upload progress, file list with review status badges

---

1.4 Admin Dashboard

The admin dashboard is the operational control centre, protected behind HMAC-SHA256 signed session cookies.

Authentication: - Login at `/admin/login` with email and password (controlled by `ADMIN_EMAIL` and `ADMIN_PASSWORD` environment variables) - 7-day session cookie — stays logged in across browser restarts - `proxy.ts` middleware intercepts every `/admin/*` request and verifies the cookie before the page loads - Logout button in sidebar footer clears the cookie and redirects to login

Admin Pages:

Page	Route	Features
Overview	`/admin`	KPI cards (inquiries this month, consultations this week, conversion rate, active clients), recent inquiries list, upcoming consultations panel, conversion funnel visualisation
Inquiries	`/admin/inquiries`	Paginated table (20/page), live search by name/email/phone, filter by status/service/priority, inline status dropdown that saves live without page reload, CSV export download
Consultations	`/admin/consultations`	Paginated table, stat cards (today/upcoming/completed/no-

Page	Route	Features
		show rate), filter by status, Join Meeting links for scheduled calls
Documents	`/admin/documents`	All uploaded client documents, filter by review status, one-click Approve and Request Resubmission actions
Analytics	`/admin/analytics`	Live Supabase data — weekly bar chart, conversion funnel, service breakdown, top destinations, traffic source breakdown, CSV export
WhatsApp Inbox	`/admin/inbox`	Full live WhatsApp chat (detailed in section 1.7 below)
Settings	`/admin/settings`	Team members table, email sequences list, integrations connection status panel, notification preferences, company information form

---

1.5 Payment Processing (Paystack)

Route: `/payment/callback` API: `POST /api/payments/initiate`, `GET /api/payments/verify` Webhook: `POST /api/webhooks/paystack`

Payment flow: 1. Admin creates a payment record for a client in the database 2. Client sees a “Pay Now” button on their dashboard payment tab 3. Client clicks Pay — frontend calls `/api/payments/initiate` 4. Backend calls Paystack API, receives a unique `authorization_url` 5. Client is redirected to Paystack’s hosted payment page 6. Client pays with any Nigerian card, bank transfer, or USSD 7. Paystack redirects back to `/payment/callback?reference=xxx` 8. Callback page calls `/api/payments/verify` to confirm with Paystack 9. Database is updated: payment status set to `paid`, client profile `payment_status` recalculated 10. Paystack also sends a webhook to `/api/webhooks/paystack` as a backup confirmation

Security: Webhook signature is verified using HMAC-SHA512 with the Paystack secret key. Invalid signatures return 401.

Currency: Nigerian Naira (NGN). Amounts stored in Naira; converted to kobo (×100) for Paystack API calls automatically.

1.6 Document Upload and Review System

Upload API: `POST /api/documents/upload` Admin review: `POST /api/admin/documents/[id]/status` Storage: Supabase Storage (`documents` bucket)

Student upload flow: 1. Student logs into dashboard → Documents tab 2. Selects document type from dropdown (10 categories) 3. Selects file (PDF, JPG, PNG, DOCX — max 10 MB) 4. File uploads to Supabase Storage under `client_id/doctype_timestamp.ext` 5. Database record created with `status: pending_review` 6. File appears instantly in the document list

Admin review flow: 1. Admin opens `/admin/documents` 2. Sees all uploads with client name, document type, file size, upload date 3. Filter by status: Pending Review, Approved, Rejected, Needs Resubmission 4. One-click Approve or Request Resubmission (POST form action, no JavaScript required)

1.7 WhatsApp Business Inbox

Route: `/admin/inbox` Webhook: `GET /api/webhooks/whatsapp` (Meta verification), `POST /api/webhooks/whatsapp` (incoming messages) Send API: `POST /api/whatsapp/send` Conversations: `GET /api/whatsapp/conversations`, `GET /api/whatsapp/conversations/[id]`, `PATCH /api/whatsapp/conversations/[id]`

This is the most significant new feature. The admin team can now receive and reply to WhatsApp messages from inside the MakeoverArena dashboard, without leaving the app or touching a phone.

How it works:

```
Customer sends WhatsApp message
        ↓
Meta Cloud API
        ↓
POST /api/webhooks/whatsapp (HMAC-SHA256 verified)
        ↓
Supabase — upserts conversation, inserts message, increments unread count
        ↓
Supabase Realtime — pushes change to admin browser instantly
        ↓
Admin sees message appear in inbox in real time
        ↓
Admin types reply and clicks Send
        ↓
POST /api/whatsapp/send → Meta Cloud API → customer's WhatsApp
```

Inbox UI features: - WhatsApp-accurate visual design (green #25D366, chat bubbles, dark green sent / white received) - Real-time message delivery — new messages appear without refreshing (Supabase Realtime WebSocket) - Double-tick delivery receipts: single tick (sent), grey double tick (delivered), blue double tick (read) - Optimistic UI — sent messages appear immediately; rolled back if Meta API rejects - Conversation list with last message preview, unread count badges, relative timestamps - Search conversations by name, phone number, or message content - Status filter tabs: All / Open / Pending / Resolved - Mark conversations as Open, Pending, Resolved, or Spam - Reopen resolved conversations - "Open in WhatsApp" link opens the conversation directly in WhatsApp Web - Mobile responsive — conversation list and chat thread swap on mobile - Media message support — images (with thumbnail), documents (filename + icon), audio (voice message label), video - Date separators in chat thread (Today, Yesterday, full date for older) - Keyboard shortcut: Enter to send, Shift+Enter for new line

Security: All API routes require valid admin session cookie. Webhook verifies Meta's `x-hub-signature-256` HMAC-SHA256 signature. Invalid signatures return 401.

1.8 AI Chatbot

The floating AI assistant appears on all public pages and answers visitor questions instantly.

Widget features: - Floating button (bottom-right) with unread badge and green online indicator - Smooth open/close animation - Quick-reply buttons for the 4 most common questions - Typing indicator (three animated dots) - Full conversation history within the browser session

Backend (`POST /api/chatbot`): - Uses OpenAI `gpt-4o-mini` with a custom system prompt containing MakeoverArena's services, pricing, success rates, and escalation rules - Rate limited — 30 messages per minute per IP (Upstash Redis) - Saves every conversation to `chat_conversations` table in Supabase with session ID - Graceful fallback — if OpenAI key is not configured, returns a friendly 503 message

System prompt covers: services, pricing (₦150,000–₦500,000), 92% success rate, 3–6 month timeline, 200+ partner universities, escalation to human, booking CTA.

1.9 Email Automation System

Nine email templates are built using React Email and sent via the Resend API. All emails are logged to the `email_logs` database table for a full audit trail.

Template	Trigger	Recipients
Inquiry Confirmation	Student submits apply form	Student
Admin Notification	Student submits apply form	Admin team
Inquiry Follow-up	48 hours after inquiry, if status is still "new"	Student
Consultation Confirmation	Cal.com booking created	Student
Consultation Cancellation	Cal.com booking cancelled	Student
Consultation Reminder	24h and 1h before scheduled call	Student
Nurture Week 1	7 days after inquiry	Student
Nurture Week 2	14 days after inquiry	Student
Nurture Week 3	21 days after inquiry	Student
Nurture Week 4	28 days after inquiry	Student

Three Supabase Edge Functions automate the timed emails: - `send-followups` — runs daily at 9am, sends 48hr follow-up to uncontacted inquiries - `send-reminders` — runs hourly, sends 24hr and 1hr consultation reminders - `handle-no-shows` — runs daily, marks missed consultations and sends rescue emails

---

1.10 Consultation Booking (Cal.com)

Route: `/book` Webhook: `POST /api/webhooks/calcom`

The booking page embeds the Cal.com scheduling interface directly. When a student books:

1. Cal.com webhook fires to `/api/webhooks/calcom`
2. System creates a `consultations` database record
3. Links the consultation to the student's existing inquiry by email
4. Updates inquiry status to `consultation_booked`
5. Sends consultation confirmation email to student
6. Sends booking notification email to admin

Rescheduling and cancellation are handled automatically — rescheduled consultations update the date/time and reset reminder flags; cancelled consultations notify the student by email.

1.11 Database Schema

11 tables in PostgreSQL via Supabase, all with Row Level Security (RLS) enabled:

Table	Purpose
`inquiries`	All student applications. 30+ columns. Indexed by status, date, email, assigned staff

Table	Purpose
`consultations`	Booking records from Cal.com. Linked to inquiries
`staff_profiles`	Admin, consultant, and viewer role management
`email_logs`	Every email sent — template, recipient, Resend ID, status, open/click tracking
`chat_conversations`	AI chatbot sessions with full message history (JSONB)
`analytics_events`	Custom event tracking for form steps, bookings, and chatbot interactions
`client_profiles`	Onboarded students with payment status and advisor assignment
`application_tracking`	Per-client application stages (JSONB stages array) with institution and country
`payments`	Payment records linked to clients — Paystack reference, status, timestamps
`documents`	Uploaded file records with storage path, review status, and reviewer
`whatsapp_conversations`	One row per WhatsApp contact — last message, unread count, status
`whatsapp_messages`	Every WhatsApp message — inbound and outbound with delivery status

Two SQL views power the analytics dashboard: - `daily_inquiry_stats` — daily counts, conversions, and conversion rate - `service_breakdown` — per-service totals, conversions, and conversion rate

---

1.12 Security Implemented

- **Admin authentication:** HMAC-SHA256 signed httpOnly session cookies. Cookie tied to `ADMIN_EMAIL` + `ADMIN_PASSWORD` hash — changing the password instantly invalidates all sessions.
- **Staff portal guard:** `/staff/dashboard` and all staff routes require the admin session cookie. Previously unprotected; now redirects to admin login.
- **Proxy middleware:** All `/admin/*` routes pass through `proxy.ts` which verifies the session before any page code runs.
- **Paystack webhooks:** HMAC-SHA512 signature verification on every incoming webhook event.
- **WhatsApp webhooks:** HMAC-SHA256 `x-hub-signature-256` verification on every incoming webhook event.
- **Cal.com webhooks:** HMAC-SHA256 `x-cal-signature-256` verification.
- **Input validation:** Zod v4 schema validation on all API inputs before any database write.
- **Rate limiting:** Upstash Redis sliding window on the apply form (5/hour per IP) and chatbot (30/minute per IP). Graceful fallback — requests pass through if Redis is unavailable.
- **RLS policies:** Every Supabase table has Row Level Security. Public can only insert inquiries and chatbot sessions. Clients can only read their own data. Staff can read everything.

1.13 Bug Fixes Applied

The following bugs were identified and corrected during the development period:

1. **Success screen never shown** — The apply form was redirecting to `/login` immediately after submission, before the success screen could render. Fixed by removing the redirect and showing the success screen with appropriate CTAs.

2. **Form errors swallowed** — API error messages from failed submissions were being discarded. Fixed to parse and display the actual API error in a toast notification.
3. **Staff portal publicly accessible** — Any person could access `/staff/dashboard` without authentication. Fixed by adding a session cookie guard to the staff layout.
4. **All emails from sandbox address** — The `from` email address was hardcoded to `onboarding@resend.dev`. Fixed to read from the `EMAIL_FROM` environment variable, allowing the real domain to be configured without a code change.
5. **Booking cancellation silent** — When a student cancelled a Cal.com consultation, no email was sent. Fixed to send a branded cancellation notification.
6. **Footer contact details hardcoded** — Phone number and social media links were static placeholder values. Fixed to read from environment variables.
7. **Admin inquiries search disabled** — The search input had a `disabled` attribute. Fixed and wired to a Supabase `ilike` query against name, email, and phone.
8. **Inline status changes required page refresh** — Admins could not update inquiry status without reloading the page. Fixed with `InquiryStatusSelect` client component and `PATCH /api/admin/inquiries/[id]/status`.
9. **Analytics page showed hardcoded mock data** — All charts and KPI cards displayed made-up numbers. Fixed to query live Supabase data.
10. **Missing database tables** — `client_profiles`, `application_tracking`, `payments`, `documents`, `whatsapp_conversations`, and `whatsapp_messages` were referenced in code but did not exist in the database migration. All six tables added with full RLS policies.
11. **CSV export non-functional** — The Export CSV button had no handler. Fixed with a real API route that streams the CSV file.
12. **Login/signup pages ignored query parameters** — After applying, students were directed to `/signup` but their email was not prefilled. Fixed; both pages now accept and apply `?email=` and `?redirect=` parameters.

Part 2 — Integration Credentials Required

Every integration listed below requires credentials or configuration that only the business owners can provide. The application code is fully built; it is waiting on these keys to become operational.

Integration 1 — Supabase (Database)

Status: Connected · Migrations NOT yet applied

Supabase is the database that stores everything — inquiries, consultations, clients, payments, documents, emails, WhatsApp messages, chatbot history. The Supabase project is already created and linked. However, the database schema has never been applied. Until the migrations are pushed, the application will silently fail on every database operation.

What is needed from the owners: The database password, which is separate from the Supabase account login.

How to find it: 1. Go to supabase.com and sign in 2. Click on the **MakeoverArena** project 3. Go to **Settings** (gear icon in the left sidebar) 4. Click **Database** 5. Scroll to **Database password** — click **Reveal** to see it 6. Copy the password

What to do with it: Run this command in the project terminal (replace `YOUR_PASSWORD` with the actual password):

```
supabase db push --linked --password YOUR_PASSWORD
```

This will create all 12 tables, RLS policies, indexes, and views in the remote database. It takes under 30 seconds.

Also required from Supabase:

One manual step that cannot be done via code: - Go to your Supabase project → **Storage** → **New Bucket** - Name: `documents` - Public: **No** (private) - Click Create

This bucket stores all student-uploaded files. Without it, document uploads will fail.

Integration 2 — Resend (Email Delivery)

Status: API key present · Sending from sandbox address · Will land in spam

Resend is the email delivery service. The API key is already configured and emails are being sent. However, all emails currently show `onboarding@resend.dev` as the sender, which marks them as promotional and will land in students' spam folders. To send from `noreply@makeoverarena.com`, a sending domain must be verified.

What is needed from the owners: 1. Access to the DNS records for `makeoverarena.com` (via domain registrar — GoDaddy, Namecheap, Cloudflare, etc.) 2. Access to the Resend account (already exists — email: j.oarowolo@gmail.com)

Steps to verify the domain: 1. Go to resend.com → sign in → **Domains** 2. Click **Add Domain** → enter `makeoverarena.com` 3. Resend will show 3–4 DNS records (SPF, DKIM, DMARC) 4. Log into your domain registrar (GoDaddy/Namecheap/Cloudflare) 5. Add each DNS record exactly as shown by Resend 6. Back in Resend, click **Verify** — DNS propagation takes 5–30 minutes 7. Once verified, update the environment variable:

```
EMAIL_FROM=MakeoverArena <noreply@makeoverarena.com>
```

Expected cost: Free on Resend's free tier (3,000 emails/month). The current usage will comfortably fit within this.

Integration 3 — Paystack (Payments)

Status: Not configured · No keys · Payments will return "not configured" error

Paystack processes all student payments — consultation fees, service packages, and any other charges. Nigerian and African cards, bank transfers, and USSD payments are all supported.

What is needed from the owners: 1. A Paystack business account 2. Test API keys (for testing before launch) 3. Live API keys (for real payments after launch)

How to create and obtain keys: 1. Go to dashboard.paystack.com and create an account 2. Complete business verification (requires CAC registration number or BVN for individuals) 3. Go to **Settings** → **API Keys &**

Webhooks 4. You will see two sets of keys: **Test** and **Live** 5. Copy: - `Secret Key` (starts with `sk_test_...` for test, `sk_live_...` for live) - `Public Key` (starts with `pk_test_...` for test, `pk_live_...` for live)

Configure in `.env.local`:

```
PAYSTACK_SECRET_KEY=sk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY=pk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Configure webhook in Paystack dashboard: 1. Settings → API Keys & Webhooks → Webhooks 2. Add webhook URL: `https://makeoverarena.com/api/webhooks/paystack` 3. No additional secret is needed — Paystack uses your secret key for HMAC verification

Testing payments: Use Paystack test card number: `4084 0840 8408 4081` with any future expiry and CVV `408`

Note: Switch to live keys before the official launch. Never mix test and live keys.

Integration 4 — Cal.com (Consultation Booking)

Status: Not configured · Booking page shows placeholder message

Cal.com manages the free 30-minute consultation scheduling. Students pick a date and time; Cal.com handles calendar sync, reminders, and video call links.

What is needed from the owners: 1. A Cal.com account (free plan is sufficient) 2. A configured event type 3. Webhook secret and URL configured

Setup steps:

Step 1 — Create account: Go to cal.com → Sign up with the business email → Complete profile

Step 2 — Configure event type: 1. Click **New Event Type** 2. Title: "Free Study Abroad Consultation" 3. Duration: 30 minutes 4. Description: "A free 30-minute session with a MakeoverArena expert to discuss your study abroad goals." 5. Video conferencing: Connect Google Meet or Zoom 6. Under **Advanced** → enable confirmation emails 7. Click **Save** 8. Copy the event URL (e.g. `https://cal.com/makeoverarena/consultation`)

Step 3 — Set environment variable:

```
NEXT_PUBLIC_CALCOM_URL=https://cal.com/makeoverarena/consultation
```

Step 4 — Configure webhook: 1. In Cal.com → Settings → Developer → Webhooks 2. Add new webhook: - URL: `https://makeoverarena.com/api/webhooks/calcom` - Events: BOOKING_CREATED, BOOKING_RESCHEDULED, BOOKING_CANCELLED - Secret: choose any strong random string (e.g. use a password generator) 3. Copy the secret and set:

```
CALCOM_WEBHOOK_SECRET=your_chosen_secret_here
```

Integration 5 — OpenAI (AI Chatbot)

Status: Not configured · Chatbot returns "unavailable" message to visitors

The AI chatbot widget uses OpenAI's `gpt-4o-mini` model. Without an API key, it returns a service unavailable error.

What is needed from the owners: An OpenAI API key and a funded account (minimum $5 credit to start).

How to obtain: 1. Go to platform.openai.com → sign in or create account 2. Click on the organisation name (top right) → **View API Keys** 3. Click **Create new secret key** 4. Name it: "MakeoverArena Chatbot" 5. Copy the key (it starts with `sk-`) — this is shown only once 6. Go to **Billing** → Add a credit card and purchase at least $5 credit

Configure:

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Estimated cost: At current traffic levels, `gpt-4o-mini` will cost approximately $3–8 per month. The model is chosen specifically for its cost efficiency.

Integration 6 — WhatsApp Business Cloud API (Live Inbox)

Status: Webhook built · Keys not configured · Inbox will show empty

The WhatsApp inbox allows the admin team to send and receive WhatsApp messages from inside the dashboard in real time. This uses Meta's official WhatsApp Business Cloud API.

What is needed from the owners: 1. A Meta Business account 2. The WhatsApp Business number registered with Meta 3. A Meta developer app with WhatsApp product added

This is the most involved setup. Follow each step carefully.

Step 1 — Create Meta Business Account 1. Go to business.facebook.com 2. Click **Create Account** and complete business verification 3. Business name: MakeoverArena 4. You will need a Facebook account to continue

---

Step 2 — Create Developer App 1. Go to developers.facebook.com 2. Click **My Apps** → **Create App** 3. Select **Business** as the app type 4. App name: "MakeoverArena" 5. Connect it to your Business Manager account created in Step 1 6. Click **Create App**

Step 3 — Add WhatsApp Product 1. On the app dashboard, scroll to find **WhatsApp** and click **Set Up** 2. You will be taken to the WhatsApp API setup page

---

Step 4 — Get Test Credentials On the WhatsApp API setup page: 1. Under **Step 1 — Select phone numbers**, you will see a test phone number Meta provides. Note its **Phone Number ID** — a long numeric string like `123456789012345` 2. Under **Step 2 — Send and receive messages**, there will be a temporary **Access Token** (expires in 24 hours — fine for testing)

To get a **permanent access token**: 1. Go to business.facebook.com → **Settings** → **System Users** 2. Create a System User with **Admin** role 3. Click **Generate New Token** 4. Select your app, grant `whatsapp_business_messaging` and `whatsapp_business_management` permissions 5. Copy the token — this one does not expire

---

Step 5 — Add Your Real Business Number 1. On the WhatsApp API setup page → **Step 1** → **Add phone number** 2. Enter the business WhatsApp number (+234…) 3. Verify via SMS or phone call 4. This becomes your `WHATSAPP_PHONE_NUMBER_ID`

Step 6 — Configure the Webhook 1. On the app dashboard → WhatsApp → **Configuration** → **Webhooks** 2. Click **Edit** 3. **Callback URL:** `https://makeoverarena.com/api/webhooks/whatsapp` 4. **Verify Token:** `makeoverarena_wa_verify_2024` (this is already set in the code) 5. Click **Verify and Save** 6. Under **Webhook fields**, enable the `messages` field

---

Step 7 — Get App Secret 1. App dashboard → **Settings** → **Basic** 2. Scroll to **App Secret** → click **Show** 3. Copy the secret

---

Step 8 — Set All Environment Variables

```
WHATSAPP_ACCESS_TOKEN=your_permanent_system_user_token
WHATSAPP_PHONE_NUMBER_ID=your_phone_number_id_here
WHATSAPP_WEBHOOK_VERIFY_TOKEN=makeoverarena_wa_verify_2024
WHATSAPP_APP_SECRET=your_app_secret_here
NEXT_PUBLIC_WHATSAPP_NUMBER=2348XXXXXXXXX
```

Cost: WhatsApp Cloud API charges per conversation (24-hour window): - Service conversations (customer initiates): Free (first 1,000/month free, then ~$0.015–0.04 each) - Business-initiated messages: ~$0.04–0.08 each - At current volumes, budget $10–30/month

---

Integration 7 — Upstash Redis (Rate Limiting)

Status: Not configured · Rate limiting is disabled · Abuse possible

Upstash Redis provides rate limiting for the apply form and chatbot. Without it, someone could spam the form or chatbot. The system gracefully allows all requests through when Redis is not configured — it does not break — but the protection is absent.

What is needed: A free Upstash account and a Redis database.

Steps (takes 5 minutes): 1. Go to upstash.com → Sign up with Google or email 2. Click **Create Database** 3. Name: `makeoverarena-ratelimit` 4. Region: `eu-west-1` (Ireland) or `us-east-1` depending on where Vercel deployment will be 5. Click **Create** 6. In the database details page, find: - **REST URL** — copy it - **REST Token** — click to reveal and copy

Configure:

```
UPSTASH_REDIS_REST_URL=https://xxxxxxxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=AXxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Cost: Free tier — 10,000 commands/day. Sufficient for the current traffic level.

Integration 8 — Google Analytics 4 (Website Analytics)

Status: Script injected · Measurement ID is placeholder · No data being collected

Google Analytics 4 tracks pageviews, user behaviour, and custom events (form steps, chatbot opens, booking completions).

What is needed: A GA4 Measurement ID for the domain `makeoverarena.com`.

Steps: 1. Go to analytics.google.com 2. Click **Admin** (gear icon) → **Create Property** 3. Property name: MakeoverArena 4. Reporting time zone: Africa/Lagos (WAT) 5. Currency: Nigerian Naira (₦) 6. Click **Next** → Business information → **Create** 7. Under **Data collection** → **Web** → enter `https://makeoverarena.com` and stream name "MakeoverArena Web" 8. Copy the **Measurement ID** (format: `G-XXXXXXXXXX`)

Configure:

```
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX
```

Cost: Free.

Integration 9 — Admin Password & Email

Status: Default placeholder password · Must be changed before deployment

The admin dashboard password is currently set to `change-me-before-deploy`. This must be changed to a strong, unique password before the application goes live.

What to set:

```
ADMIN_EMAIL=your-real-admin-email@domain.com
ADMIN_PASSWORD=choose-a-strong-password-with-uppercase-numbers-and-symbols
```

Requirements for the password: - Minimum 16 characters - Mix of uppercase, lowercase, numbers, and symbols - Do not use this password on any other service - Store it in a password manager (1Password, Bitwarden, or similar)

Note: Changing `ADMIN_PASSWORD` instantly invalidates all existing admin sessions. Anyone logged in will be logged out and must re-enter the new password.

Part 3 — Pre-Launch Checklist

Once all credentials are configured, the following actions are needed in order:

Step 1 — Apply Database Migrations

```
supabase db push --linked --password YOUR_DB_PASSWORD
```

Creates all 12 database tables. **Required before any other step.**

Step 2 — Create Supabase Storage Bucket

In Supabase Dashboard → Storage → New Bucket → name: `documents` → private.

Step 3 — Change Admin Password

Update `ADMIN_EMAIL` and `ADMIN_PASSWORD` in `.env.local` to real values.

Step 4 — Configure Paystack

Add test keys. Run a test payment using card `4084 0840 8408 4081`.

Step 5 — Configure Resend Domain

Add DNS records. Verify domain. Update `EMAIL_FROM`.

Step 6 — Configure Cal.com

Create event type. Set `NEXT_PUBLIC_CALCOM_URL`. Configure webhook.

Step 7 — Configure OpenAI

Add key. Test chatbot at `/` by asking "How much does it cost?"

Step 8 — Configure WhatsApp

Follow the 8-step Meta setup. Send a test message to the business number and confirm it appears in `/admin/inbox`.

Step 9 — Configure Upstash Redis

Create database. Add URL and token. Confirm rate limiter activates on the 6th form submission.

Step 10 — End-to-End Test

Run through the full student journey: 1. Submit form at `/apply` → confirm confirmation email received 2. Book consultation at `/book` → confirm admin notified 3. Sign up at `/signup` → confirm dashboard access 4. Make test payment → confirm callback page, DB updated 5. Upload a document → confirm appears in `/admin/documents` 6. Send WhatsApp to business number → confirm appears in `/admin/inbox` 7. Reply from admin inbox → confirm received on WhatsApp

Step 11 — Deploy to Vercel

1. Push all environment variables to Vercel:

   ```
   vercel env pull  # sync local .env.local to Vercel
   ```

   Or add each variable manually at vercel.com → Project → Settings → Environment Variables

2. Deploy: `vercel --prod`

3. Configure custom domain `makeoverarena.com` in Vercel DNS settings

Step 12 — Switch Paystack to Live Mode

Replace `sk_test_...` with `sk_live_...` and `pk_test_...` with `pk_live_...`.

Step 13 — Deploy Supabase Edge Functions

```
supabase functions deploy send-followups
supabase functions deploy send-reminders
supabase functions deploy handle-no-shows
```

Part 4 — Cost Estimate (Monthly at Launch)

Service	Plan	Estimated Cost
Vercel	Hobby (free) or Pro ($20/mo)	$0–20
Supabase	Free tier (500MB, 50,000 rows)	$0
Resend	Free tier (3,000 emails/month)	$0
OpenAI	Pay-as-you-go (gpt-4o-mini)	~$5–15
Paystack	% per transaction (1.5% + ₦100 cap)	Transaction fee only
Cal.com	Free plan	$0
Upstash Redis	Free tier (10,000 commands/day)	$0
Google Analytics	Free	$0
WhatsApp Cloud API	Per conversation (~$0.02 avg)	$10–30
Total		**~$15–65/month**

Paystack charges 1.5% per transaction (capped at ₦2,000 per transaction). For a ₦200,000 service package, the Paystack fee is ₦2,000 (1%).

Summary Table — All Integrations

#	Integration	Purpose	Status	Action Required
1	**Supabase**	Database (all data)	Connected, schema not applied	Push migrations + create storage bucket
2	**Resend**	Email delivery	Working (sandbox only)	Verify makeoverarena.com domain
3	**Paystack**	Payment processing	Not configured	Create account, get API keys, configure webhook
4	**Cal.com**	Consultation booking	Not configured	Create account, configure event, set webhook
5	**OpenAI**	AI chatbot	Not configured	Create account, add API key, add billing credit
6	**WhatsApp**	Live customer chat	Not configured	Complete 8-step Meta developer setup
7	**Upstash Redis**	Rate limiting (spam protection)	Not configured	Create free database, add URL + token
8	**Google Analytics 4**	Website analytics	Script present (no ID)	Create GA4 property, add measurement ID
9	**Admin credentials**	Dashboard access	Placeholder password	Set real ADMIN_EMAIL and ADMIN_PASSWORD

Report generated: 2 June 2026 Engineering contact: Available in this session for any clarifications